

Synca Technical Whitepaper

Version 1.0 — May 2026

Confidential — For Engineering, Security, and Technical Due Diligence Use Only

1. Introduction

Synca is a lifestyle-based matchmaking platform that uses passively collected behavioral signals — primarily health, music, travel, and visual preference data — to estimate compatibility between people and propose pre-organized real-world dates.

This whitepaper describes the technical architecture, data model, matching algorithms, privacy and security design, and integration points with external systems (Apple HealthKit, Android Health Connect, Spotify, travel services, and Telegram). It is intended for technical stakeholders: investor CTOs, security reviewers, and potential integration partners.

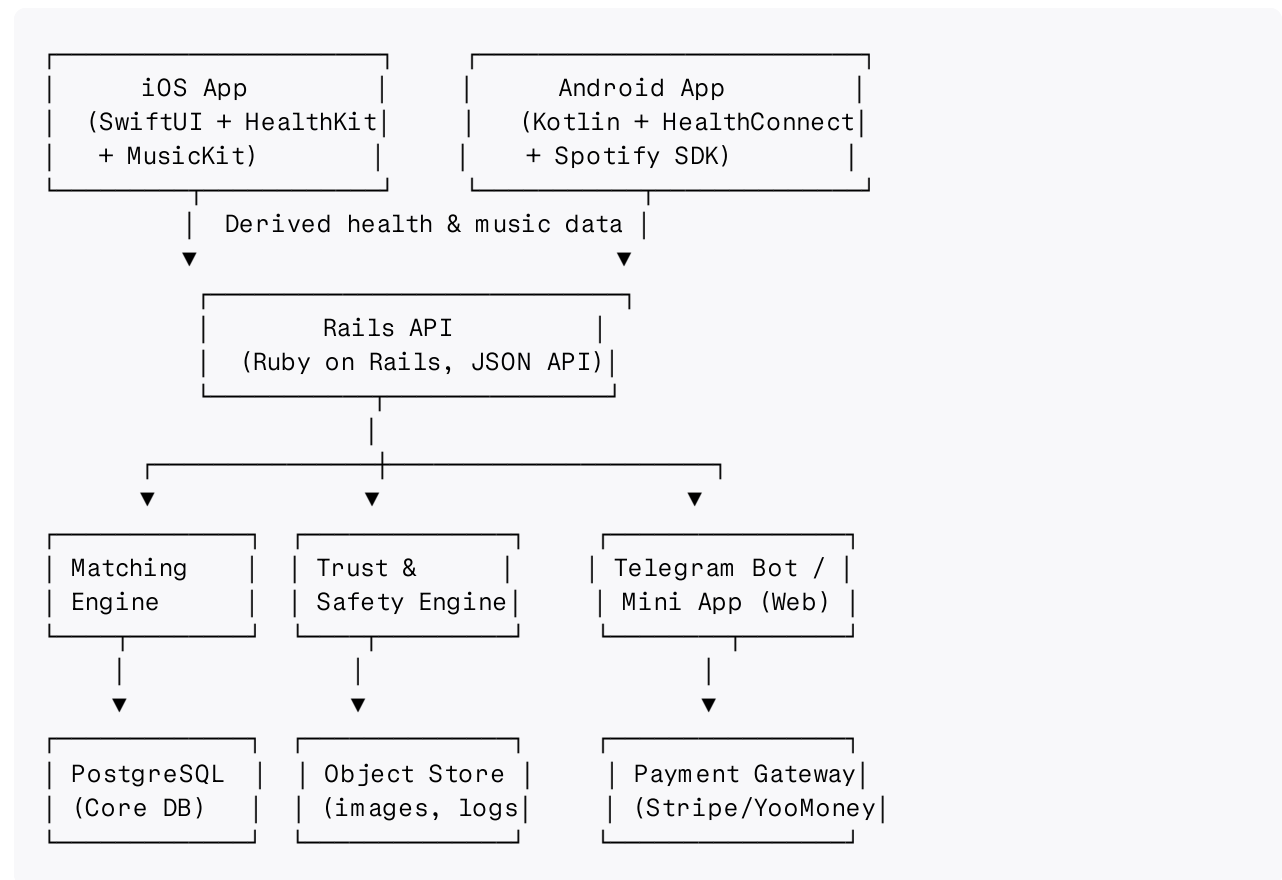
2. System Architecture Overview

2.1 High-Level Components

Synca's architecture is composed of six main components:

1. **iOS Client** — SwiftUI app using HealthKit for health data, MusicKit for Apple Music (future), and local processing of raw samples.
2. **Android Client** — Kotlin app using Health Connect for health data, Spotify SDK for music, and on-device aggregation.
3. **Rails API Backend** — central service implementing all business logic: user management, signal ingestion, matching engine, trust scoring, date proposal generation, payment integration.
4. **Compatibility Engine** — stateless service or internal Rails module computing compatibility scores, expressed as a weighted combination of multiple normalized features.
5. **Trust & Safety Engine** — pipelines and services that compute a dynamic Trust Score based on image, behavior, and cross-signal consistency.
6. **Telegram Bot / Mini App** — WebApp client integrated with Telegram Bot API, used for acquisition, notifications, and payment flows, especially in markets with restricted app store access.

2.2 Logical Architecture Diagram



The system is designed so that:

- Raw health and music samples are never transmitted — only derived aggregates.
- All client surfaces (native apps, Telegram Mini App) talk to the same Rails API.
- Trust & Safety logic can evolve independently from client releases.

3. Data Model and Schemas

This section outlines the core entities and how they relate to each other. The actual implementation is in PostgreSQL using Rails ActiveRecord.

3.1 Core Entities

User

- `id`: UUID
- `email`: string (nullable if sign-in via Apple/Google only)
- `auth_identity_type`: enum (apple, google, telegram, email)
- `created_at`, `updated_at`
- `city_id`: foreign key to City
- `birth_date`: date

- `gender:enum`
- `interested_in:enum` or multi-valued (for orientation)
- `has_wearable:boolean` (derived)

Profile

- `user_id:fk`
- `display_name:string`
- `bio:text`
- `photos:references` to Photo table
- `occupation:string`
- `lifestyle_tags:array` (derived from signals and onboarding)

City

- `id:UUID`
- `name:string` (e.g., "Moscow")
- `country_code:string`
- `timezone:string`
- `geo_bounds:polygon` or lat/lng bbox

HealthSummary (per-user, per-period)

- `id:UUID`
- `user_id:fk`
- `period_start:date`
- `period_end:date`
- `chronotype:enum` (morning, intermediate, evening)
- `avg_sleep_start_local:time`
- `avg_sleep_end_local:time`
- `sleep_stability_index:float` (0–1)
- `activity_level:enum` (low, medium, high)
- `peak_activity_window_start:time`
- `peak_activity_window_end:time`
- `personal_space_index:float` (variance of time spent alone vs social)
- `recovery_index:float` (HRV and RHR-normalized when available)
- `source:enum` (healthkit, health_connect, manual)

MusicProfile

- `id:UUID`
- `user_id:fk`

- `provider:enum` (spotify, apple_music, lastfm)
- `energy_score:float` (0–1)
- `valence_score:float` (0–1)
- `genre_diversity_index:float` (0–1)
- `listening_variance_by_hour:jsonb` (24-bin distribution)
- `updated_at:timestamp`

TravelProfile

- `id:UUID`
- `user_id:fk`
- `trip_frequency_per_year:float`
- `avg_trip_distance_km:float`
- `novelty_preference_index:float` (0–1)
- `preferred_trip_duration_days:float`
- `home_stability_index:float` (time spent in home city)
- `source:enum` (preference_game, maps, travel_api)

PreferenceEmbedding

- `id:UUID`
- `user_id:fk`
- `vector:float[]` (e.g., 128-dim embedding)
- `version:integer` (model version)
- `created_at, updated_at`

TrustScore

- `id:UUID`
- `user_id:fk`
- `score:float` (0–1)
- `image_liveness_score:float`
- `image_forensics_score:float`
- `behavioral_score:float`
- `health_consistency_score:float`
- `music_consistency_score:float`
- `last_recalculated_at:timestamp`

Match

- `id:UUID`
- `user_a_id, user_b_id:fk`

- `compatibility_score`: float (0–1)
- `dimensions_breakdown`: jsonb (`{"sleep":0.82, "activity":0.76, ...}`)
- `status`: enum (proposed, accepted, declined, expired)
- `city_id`: fk

DateProposal

- `id`: UUID
- `match_id`: fk
- `venue_id`: fk (nullable for generic proposals)
- `suggested_time_window`: tsrange
- `activity_type`: enum (walk, coffee, padel, sauna, etc.)
- `status`: enum (pending, confirmed, rejected)

4. Client Applications

4.1 iOS App (SwiftUI)

Key responsibilities:

- Handle onboarding (auth, permissions, preference game).
- Read HealthKit data with explicit user consent.
- Aggregate raw health samples into derived metrics locally.
- Render health dashboard and compatibility insights.
- Communicate with Rails API via JSON/HTTPS.

HealthKit Integration:

- Permissions requested for: `HKCategoryTypeIdentifierSleepAnalysis`, `HKQuantityTypeIdentifierStepCount`, `HKQuantityTypeIdentifierActiveEnergyBurned`, optionally `HKQuantityTypeIdentifierHeartRate`, `HKQuantityTypeIdentifierHeartRateVariabilitySDNN`.
- Aggregation jobs run in background using `HKObserverQuery` and `HKAnchoredObjectQuery`.
- A local aggregator computes rolling 14–30-day metrics and uploads a new `HealthSummary` object when significant change occurs or at minimum once per day.

On-Device Aggregation Example:

- Chronotype computed by clustering sleep midpoint over the last 30 nights.
- Sleep stability index computed as 1 minus the normalized standard deviation of sleep start/end.
- Peak activity window derived from step count histogram across hours.

4.2 Android App (Kotlin + Health Connect)

Health Connect provides a unified interface for health data across Android OEMs, replacing Google Fit and standardizing API access from 2026 onward.^[^1]

Key responsibilities mirror iOS:

- Request Health Connect permissions for sleep, steps, heart rate.
- Aggregate locally; upload derived `HealthSummary`.
- Support devices where primary tracking is via third-party apps (e.g., Garmin, Xiaomi) that sync into Health Connect.

Implementation:

- Use `HealthConnectClient` to read data from `SleepSessionRecord`, `StepsRecord`, `HeartRateRecord`.
- Schedule periodic background jobs using `WorkManager` to recompute aggregates.

4.3 Telegram Bot and Mini App

Telegram is used primarily in:

- Russia, where app store payments and distribution are restricted.
- As a universal CRM and notification channel in other markets.

Bot functions:

- Registration and basic profile creation.
- Lightweight onboarding (preference questions, city, intent).
- Deep-linking to app store or direct APK/RuStore download.
- Surfacing match notifications and date proposals.
- Handling payments via Telegram Stars, YooMoney, or external payment URLs.

Mini App (WebApp) functions:

- Richer UI than simple chat — embedded web frontend served from Synca infrastructure.
- Communicates with Rails API using JWT tokens obtained from Telegram authorization.

5. Matching Engine

5.1 Design Principles

The matching engine is built around three principles:

1. **Multi-signal fusion** — combine independent behavioral signals: health, music, travel, visual preference.

2. **Explainability** — compatibility scores are decomposed into human-readable dimensions for user-facing explanations.
3. **Calibratability** — weights are adjustable globally and per-user as data accrues.

5.2 Feature Extraction

For each user, the engine maintains a feature vector broken down by domain:

Health features (H):

- `H_chronotype` (one-hot or scalar encoding)
- `H_sleep_midpoint` (normalized time)
- `H_sleep_stability` (0–1)
- `H_peak_activity_start`, `H_peak_activity_end`
- `H_activity_level`
- `H_personal_space_index`
- `H_recovery_index`

Music features (M):

- `M_energy` (0–1)
- `M_valence` (0–1)
- `M_genre_diversity`
- `M_evening_listening_intensity` (correlates with chronotype)

Travel features (T):

- `T_trip_frequency`
- `T_avg_trip_distance`
- `T_novelty_index`
- `T_home_stability`

Preference features (P):

- `P_vector` (embedding; similarity measured via cosine similarity)

5.3 Compatibility Calculation

For any pair of users (A, B), the engine computes partial compatibility scores:

- **Sleep alignment score:** similarity of `H_chronotype`, `H_sleep_midpoint`, and `H_sleep_stability`.
- **Peak activity overlap:** intersection over union of activity windows.
- **Activity level similarity:** categorical similarity with tolerance.
- **Personal space compatibility:** difference between `H_personal_space_index` values.

- Recovery compatibility: proximity in $H_{\text{recovery_index}}$.
- Travel style compatibility: distance in $T_{\text{novelty_index}}$, similarity in $T_{\text{trip_frequency}}$ and $T_{\text{home_stability}}$.
- Music profile compatibility: distance between $(M_{\text{energy}}, M_{\text{valence}})$ vectors, correlation of evening listening patterns.
- Visual preference compatibility: cosine similarity between P_{vector} of one user and visual features derived from the other's photos.

The final compatibility score is:

$$C(A, B) = \sum_i w_i \cdot s_i(A, B)$$

where w_i are configurable weights per dimension, and s_i are normalized similarity scores (0–1) for each dimension.

5.4 Per-User Weight Customization

Users can indicate which dimensions matter more to them (e.g., travel vs. activity level). The engine maintains both:

- Global weights w_i^{global}
- Per-user preference weights w_i^{user}

The effective weights for a pair are computed as:

$$w_i^{\text{eff}}(A, B) = f(w_i^{\text{global}}, w_i^{\text{userA}}, w_i^{\text{userB}})$$

where f can be a simple average or a more complex function giving extra weight to dimensions that both users prioritize.

5.5 Learning from Outcomes

As users interact, the engine collects outcome data:

- Whether both accepted a match.
- Whether they confirmed a date proposal.
- Self-reported outcomes after dates (short feedback forms).

These signals can be used to:

- Recalibrate global weights via gradient-free optimization.
- Train learning-to-rank models on top of hand-crafted scores.

In early stages, a deterministic weighted sum is preferred for transparency. As scale increases, more advanced models (e.g., gradient boosted trees or simple neural networks) can be introduced while preserving explainability.

6. Trust & Safety Infrastructure

6.1 Multi-Layer Trust Score

The Trust Score is designed as a composite metric combining several layers:

1. Image liveness and authenticity
2. Image forensics and metadata
3. Behavioral patterns
4. Cross-signal consistency

Each is computed separately and then combined into a single score.

6.2 Image Liveness and Authenticity

- Use third-party or in-house liveness detection models to ensure profile photos originate from live camera input.
- Capture multiple frames and micro-movements to prevent static-photo spoofing.
- Store only derived features, not raw liveness frames, to minimize sensitive biometric data retention.

6.3 Image Forensics and Metadata

- Detect AI-generated images (GAN artifacts, diffusion model signatures) using forensic classifiers.
- Analyze EXIF metadata where present; detect inconsistencies between claimed recency and timestamp.
- Cross-check resolution, aspect ratios, and compression patterns for signs of image reuse or editing.

6.4 Behavioral Patterns

- Monitor frequency and content of outgoing messages.
- Detect repetitive templated messages sent to large numbers of users.
- Flag external link sharing and specific patterns associated with scams.

No message content is used for advertising or recommendation; analysis is limited to fraud detection.

6.5 Cross-Signal Consistency

- Compare listening patterns with claimed or inferred chronotype.
- Detect health data that is unrealistically perfect or obviously fabricated.
- Check that photo contexts align with claimed lifestyle (e.g., sports tags without any sports context across photos may slightly lower trust).

6.6 Enforcement

Trust Score influences visibility but does not result in hard bans by default:

- High score → full visibility in matching pool.
- Medium score → slightly reduced visibility.
- Low score → significantly reduced visibility; user is prompted to complete additional verification steps.

This "selective ghosting" approach minimizes confrontational moderation while continuously pushing the ecosystem toward authenticity.

7. Privacy and Security by Design

7.1 Data Minimization

Synca's privacy model is built on explicit minimization:

- Only derived, aggregate metrics from health, music, and travel data are transmitted.
- No raw health samples (e.g., per-second heart rate) or precise travel trajectories are stored.
- Music data is reduced to aggregated features (energy, valence, diversity, time-of-day distributions) rather than detailed listening history.

7.2 Consent and Control

- Fine-grained consent: users choose which domains to share (health, music, travel) and can revoke any at any time.
- On-demand deletion: a single action in the app triggers deletion of all data in Synca's backend associated with that user, in line with GDPR "right to be forgotten" requirements.
[²]

7.3 GDPR and Special Category Data

Health data is treated as special category personal data under GDPR Article 9. Synca's approach:[²]

- Limit processing to derived statistics; no medical inference.
- Use consent as a lawful basis, with explicit, informed opt-in and clear explanations.[³]
- Appoint a Data Protection Officer (DPO) prior to European launch.

7.4 Data Residency

- EU user data stored in EU-based infrastructure.
- Russian user data stored in Russian-located infrastructure (e.g., Yandex Cloud) to satisfy local data localization laws.
- Segregated databases or logical partitions to avoid cross-jurisdiction data movement.

7.5 Transport and Storage Security

- All API communication over TLS 1.2+.
- OAuth2 / OpenID Connect for external integrations (Spotify, Google, Apple).
- Encryption at rest for sensitive tables (e.g., AES-256 for health and music profiles).
- Strict access control with role-based permissions.

8. External Integrations

8.1 Apple HealthKit

HealthKit provides a unified interface to health data on iOS devices and is widely adopted: Apple Health has tens of millions of active users globally.[⁴]

Integration design:

- Use HealthKit only from the native iOS app.
- Do not send HealthKit data to any third party besides Synca's own backend.
- Respect Apple's guidelines prohibiting use of health data for advertising.

8.2 Android Health Connect

Health Connect replaces Google Fit as the standard health data layer on Android, offering a standardized schema for sleep, activity, and biometrics.[¹]

Integration design:

- Use Health Connect from the Android app to read standardized records.
- Support bridging from OEM fitness apps that sync into Health Connect.

8.3 Spotify API

Spotify's Web API provides access to:

- Top artists, tracks, and genres.
- Audio features such as energy, valence, tempo, and danceability.

Integration design:

- Use OAuth2 to obtain access tokens with `user-top-read` and `user-read-recently-played` scopes.
- Fetch and aggregate top tracks and audio features periodically.
- Store only pre-computed aggregated features in `MusicProfile`.

8.4 Travel Services

Potential integrations include:

- Google Maps Timeline (if and when API access is feasible for third-party applications).
- Polarsteps and other travel logging services.

Given the sensitivity of this data, the initial focus is on:

- Onboarding travel preference game to approximate travel style.
- Limited, opt-in integrations where clear user value and strong privacy guarantees exist.

8.5 Telegram

Telegram integration comprises:

- Standard Bot API for message handling.
- MiniApp (WebApp) framework for embedded web-based UI.
- Payments via Telegram Payments API and/or external payment links.

All core data still flows through the Rails API; Telegram is a client and payment facilitator, not a data processor for sensitive health information.

9. Scaling and Reliability

9.1 Stateless Backend and Horizontal Scaling

- Rails API is stateless; session state handled via JWT tokens or encrypted cookies.
- Behind a load balancer, horizontal scaling is straightforward.

9.2 Database Scaling

- PostgreSQL as primary data store.
- Read replicas for analytical workloads and batch compatibility recomputation.
- Partitioning by city and/or region when user base grows.

9.3 Asynchronous Processing

- Background job system (Sidekiq or similar) for heavy tasks:
 - Recomputing compatibility scores when signals change.
 - Recalculating Trust Scores periodically.
 - Generating date proposals in batches.

9.4 Resilience

- Graceful degradation: if some signals (e.g., travel or music) are unavailable, the engine falls back to health + preference embedding only.
- Circuit breakers for external integrations (Spotify, Yandex AI) to avoid cascading failures.

10. Roadmap and Future Work

10.1 Short-Term (0–12 Months)

- Finalize iOS MVP with HealthKit integration, preference game, basic matching engine.
- Implement Telegram Bot and Mini App for Moscow launch.
- Integrate Yandex AI for photo context analysis in the Russian market.
- Roll out basic Spotify integration for music profile enrichment.

10.2 Medium-Term (12–24 Months)

- Launch Android app with Health Connect integration.
- Add travel preference game and optional travel service integrations.
- Introduce basic learning-to-rank layer over hand-crafted compatibility scores.
- Harden Trust Score with additional image forensics and behavioral heuristics.

10.3 Long-Term (24+ Months)

- Cross-signal predictive modeling: use combined health, music, and travel data to predict compatibility outcomes.
- Publish anonymized aggregate findings on lifestyle compatibility and relationship success.
- Develop API for third-party apps (e.g., gyms, wellness platforms) to integrate Synca compatibility signals into their own experiences.

11. Conclusion

Synca's technical design centers on a single idea: compatibility is best inferred from how people actually live, not how they describe themselves. The combination of HealthKit/Health Connect data, music listening behavior, travel patterns, and visual preference embeddings creates a rich, multi-dimensional profile for each user — while respecting strict privacy constraints through on-device processing and data minimization.

The architecture described in this whitepaper enables Synca to:

- Launch in complex regulatory and distribution environments (e.g., Russia via Telegram Mini Apps).
- Maintain a strong privacy and security posture suitable for GDPR and other modern data protection regimes.
- Scale to multiple cities and countries without re-architecting the core.

For investors and technical partners, this whitepaper demonstrates that Synca's differentiation is not a marketing veneer; it is embedded in the data model, the matching engine, and the privacy-first architecture.

For implementation details at code level (schemas, API contracts, client pseudocode), see the internal developer documentation repository.

References

1. [Migrating Users from Google Fit SDK to Health Connect](#) - Health Connect has replaced Google Fit as the main app for sharing data between Android health apps....
2. [What is GDPR, the EU's new data protection law?](#) - What is the GDPR? Europe's new data privacy and security law includes hundreds of pages' worth of ne...
3. [Health data and GDPR: Best practices for analytics in the EU](#) - Learn which EU regulations cover the use of health data, what responsibilities you shoulder and what...
4. [Consumer Wearables Generate 2.8 Petabytes of Health Data Annually](#) - Industry report reveals that consumer wearable devices generate 2.8 petabytes of health data annuall...